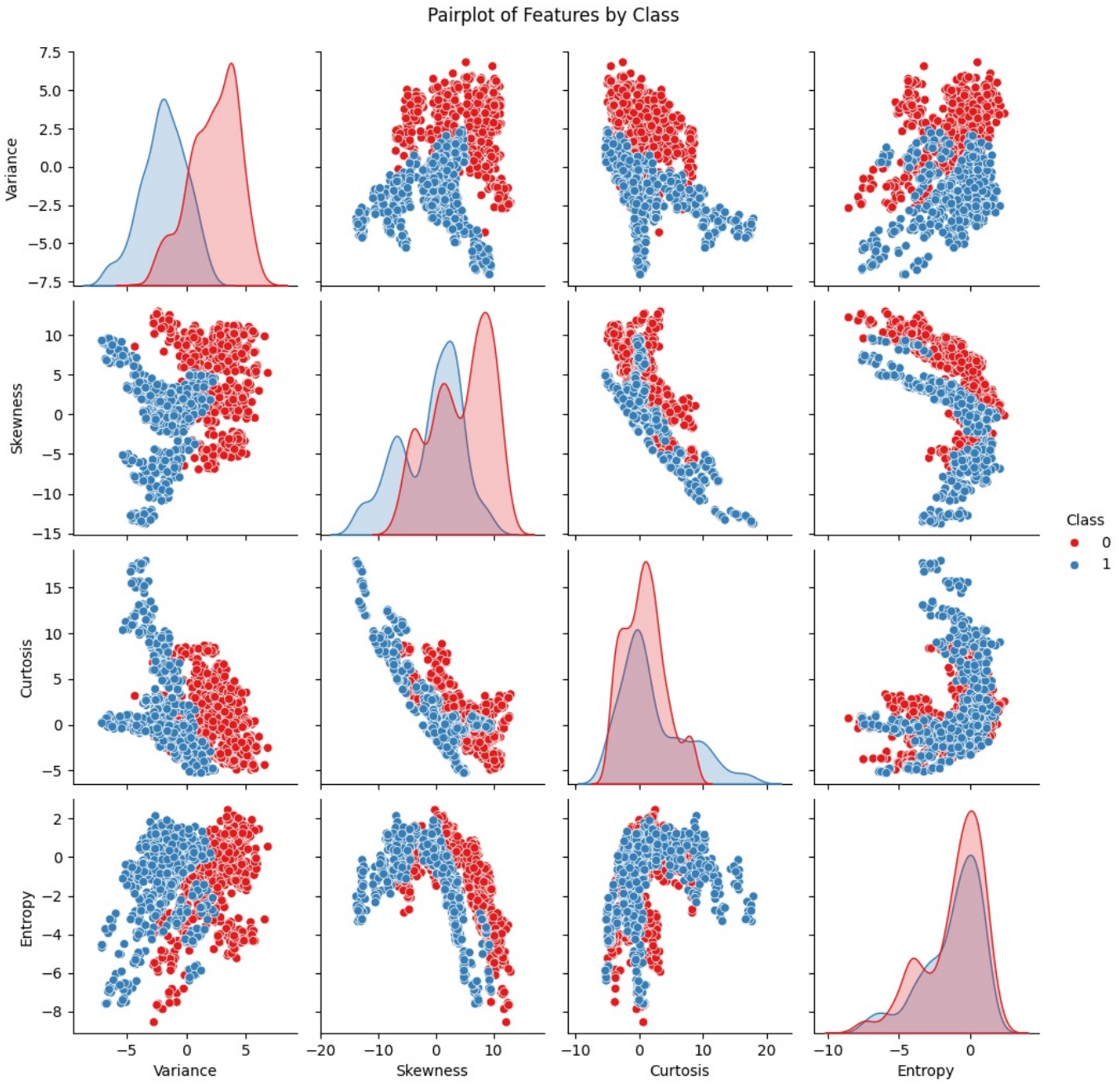
In [2]:
```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.svm import SVC
from sklearn.metrics import (classification_report, confusion_matrix, ConfusionMatrixDisplay,
                             accuracy_score, precision_score, recall_score, f1_score)
%matplotlib inline
```

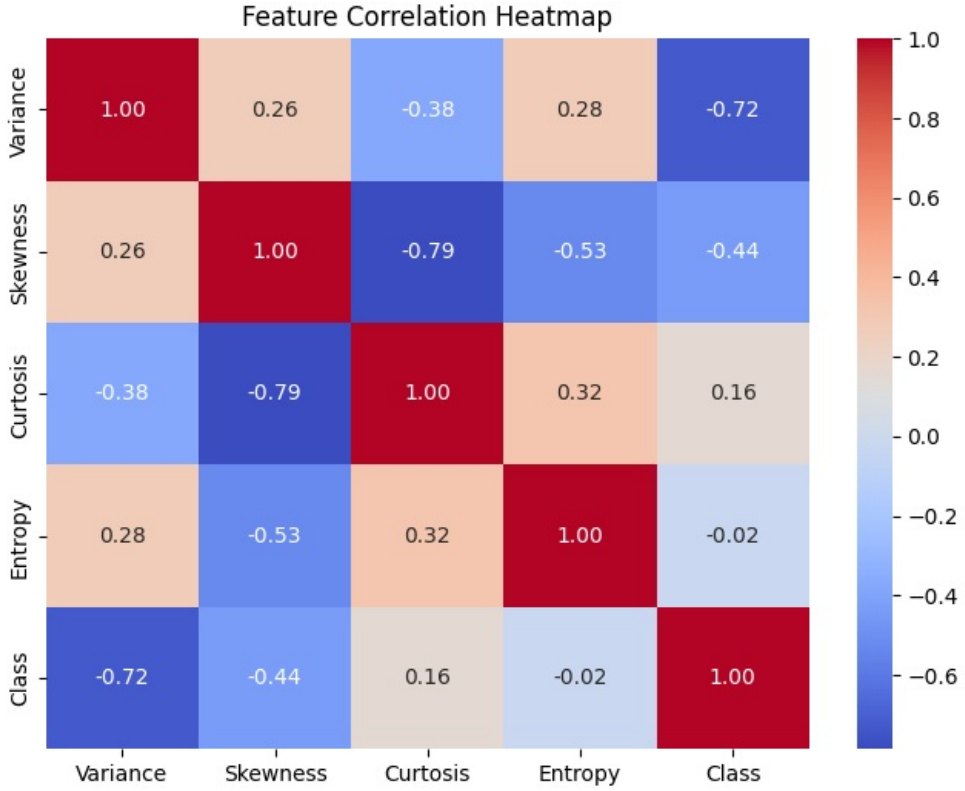
In [3]:
```
# Load data
bankdata = pd.read_csv("bill_authentication.csv")
print(f"Dataset shape: {bankdata.shape}")
display(bankdata.head())

# Prepare features and labels
X = bankdata.drop('Class', axis=1)
y = bankdata['Class']

# Split data
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.20, random_state=42)
```

```
Dataset shape: (1372, 5)
```

	Variance	Skewness	Curtosis	Entropy	Class
0	3.62160	8.6661	-2.8073	-0.44699	0
1	4.54590	8.1674	-2.4586	-1.46210	0
2	3.86600	-2.6383	1.9242	0.10645	0
3	3.45660	9.5228	-4.0112	-3.59440	0
4	0.32924	-4.4552	4.5718	-0.98880	0

In [4]:
```
import seaborn as sns

# Pairplot to see feature distributions and relationships
sns.pairplot(bankdata, hue='Class', palette='Set1')
plt.suptitle('Pairplot of Features by Class', y=1.02)
plt.show()

# Correlation Heatmap
plt.figure(figsize=(8, 6))
sns.heatmap(bankdata.corr(), annot=True, cmap='coolwarm', fmt='.2f')
plt.title('Feature Correlation Heatmap')
plt.show()
```

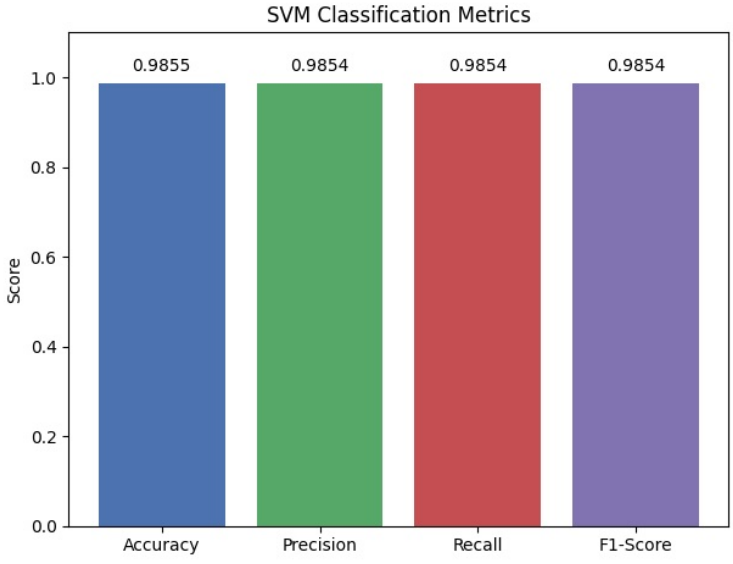

Pairplot of Features by Class
Variance
Skewness
Curtosis
Entropy
Class
0
1

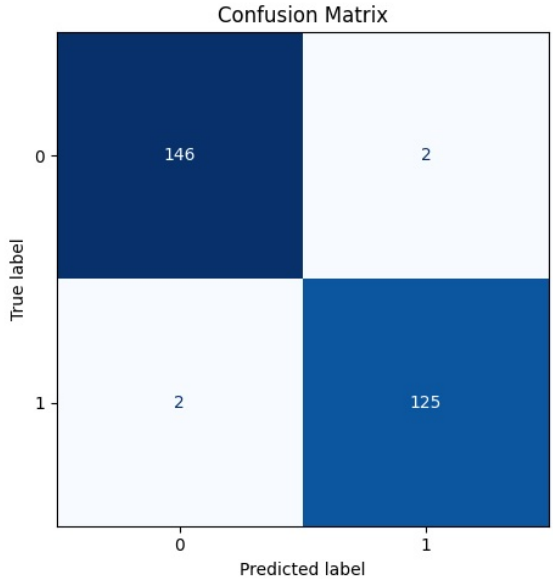

Feature Correlation Heatmap
Variance
Skewness
Curtosis
Entropy
Class
1.00 0.26 -0.38 0.28 -0.72
0.26 1.00 -0.79 -0.53 -0.44
-0.38 -0.79 1.00 0.32 0.16
0.28 -0.53 0.32 1.00 -0.02
-0.72 -0.44 0.16 -0.02 1.00

In [5]:
```
# Train SVM model
svclassifier = SVC(kernel='linear')
svclassifier.fit(X_train, y_train)

# Predict
y_pred = svclassifier.predict(X_test)
```

In [6]:
```
# Evaluate Model
print("Classification Report:")
print(classification_report(y_test, y_pred))

accuracy = accuracy_score(y_test, y_pred)
precision = precision_score(y_test, y_pred, average='macro')
recall = recall_score(y_test, y_pred, average='macro')
f1 = f1_score(y_test, y_pred, average='macro')

# Create a figure with two subplots side-by-side
fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(12, 5))

# Plot 1: Classification Metrics Bar Chart
metrics = ['Accuracy', 'Precision', 'Recall', 'F1-Score']
values = [accuracy, precision, recall, f1]
bars = ax1.bar(metrics, values, color=['#4C72B0', '#55A868', '#C44E52', '#8172B2'])
ax1.set_ylim(0, 1.1)
ax1.set_title('SVM Classification Metrics')
ax1.set_ylabel('Score')

for bar in bars:
    yval = bar.get_height()
    ax1.text(bar.get_x() + bar.get_width()/2, yval + 0.02, round(yval, 4), ha='center', va='bottom')

# Plot 2: Confusion Matrix Heatmap
cm = confusion_matrix(y_test, y_pred)
disp = ConfusionMatrixDisplay(confusion_matrix=cm)
disp.plot(cmap='Blues', ax=ax2, colorbar=False)
ax2.set_title('Confusion Matrix')

plt.tight_layout()
plt.show()
```

```
Classification Report:
              precision    recall  f1-score   support

           0       0.99      0.99      0.99       148
           1       0.98      0.98      0.98       127

    accuracy                           0.99       275
   macro avg       0.99      0.99      0.99       275
weighted avg       0.99      0.99      0.99       275
```





In [7]:
```
from sklearn.decomposition import PCA

# Reduce to 2 dimensions for visualization using PCA
pca = PCA(n_components=2)
X_train_pca = pca.fit_transform(X_train)
X_test_pca = pca.transform(X_test)

# Train a new SVM on the 2D data for visualization
svm_2d = SVC(kernel='linear')
```

```
svm_2d.fit(X_train_pca, y_train)

# Create a mesh grid
h = .02  # step size in the mesh
x_min, x_max = X_train_pca[:, 0].min() - 1, X_train_pca[:, 0].max() + 1
y_min, y_max = X_train_pca[:, 1].min() - 1, X_train_pca[:, 1].max() + 1
xx, yy = np.meshgrid(np.arange(x_min, x_max, h),
                     np.arange(y_min, y_max, h))

Z = svm_2d.predict(np.c_[xx.ravel(), yy.ravel()])
Z = Z.reshape(xx.shape)

plt.figure(figsize=(10, 8))
plt.contourf(xx, yy, Z, cmap=plt.cm.coolwarm, alpha=0.8)

# Plot training points
scatter = plt.scatter(X_train_pca[:, 0], X_train_pca[:, 1], c=y_train, cmap=plt.cm.coolwarm, edgecolors='k')
plt.title('SVM Decision Boundary (PCA Reduced 2D)')
plt.xlabel('First Principal Component')
plt.ylabel('Second Principal Component')
plt.colorbar(scatter, ticks=[0, 1], label='Class')
plt.show()
```